HW #4.3: First Data Analysis QMD File

Junbo Zhao

2026-04-13

Table of contents

1 Busiest Airports Analysis **2**
1.1 Data Overview . . . 2
1.2 Visualization . . . 2
1.3 Analysis . . . 2

2 Monte Carlo Numerical Integration **5**
2.1 Method Overview . . . 5
2.2 Small Multiples Visualization . . . 5
2.3 Analysis . . . 5

3 Planning and Prompting GenAI Tools **7**
3.1 Plan . . . 7
3.1.1 Goals . . . 7
3.1.2 Needs . . . 7
3.1.3 Steps . . . 7
3.1.4 Tidied and Wrangled Data . . . 8
3.2 Plan-Informed Results . . . 9
3.3 Generic Prompt Results . . . 11
3.4 Comparison: Plan-Informed vs. Generic . . . 11

4 Reflection **14**

Appendix A: GenAI Usage **15**

Appendix B: Code **16**

1 Busiest Airports Analysis

1.1 Data Overview

For this section I looked at annual passenger traffic at six major airports from 2020 to 2025. The airports are Hartsfield-Jackson Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Dubai International (DXB), Los Angeles International (LAX), and London Heathrow (LHR). They cover four different continents so it gives a decent snapshot of how global air travel has changed since COVID. All numbers are in millions and I got them from publicly available airport statistics and Wikipedia.

Table 1: Annual Passenger Traffic (Millions) at Six Major Airports, 2020–2025

Airport	Name	2020	2021	2022	2023	2024	2025
ATL	Hartsfield-Jackson Atlanta	42.9	75.7	93.7	104.7	108.1	106.3
FRA	Frankfurt Airport	18.8	24.8	48.9	59.4	61.6	63.2
PKX	Beijing Daxing	16.0	25.7	17.5	39.4	52.0	53.6
DXB	Dubai International	25.9	29.1	66.1	86.9	92.3	95.2
LAX	Los Angeles International	28.8	41.9	65.9	75.0	76.6	73.7
LHR	London Heathrow	22.1	19.4	61.6	79.2	83.9	84.5

1.2 Visualization

1.3 Analysis

The first thing that jumps out from Table 1 and Figure 1 is how badly 2020 hit every airport on this list. Atlanta went from normally having over 100 million passengers down to just 42.9 million. Heathrow fell to 22.1 million and Frankfurt was even lower at 18.8 million. If you look at the chart all six lines just drop straight down in 2020, so the pandemic really did wreck air travel across the board.

What's more interesting though is how differently each airport recovered after that. Dubai had the most dramatic comeback—it went from 29.1 million in 2021 to almost 87 million by 2023, which is insane. By 2025 it reached 95.2 million and set a new record. I think it's because Dubai is such a huge connecting hub for international flights so when borders reopened, transit traffic flooded back in. Atlanta got back over 100 million by 2023 too, but then it actually dipped slightly to 106.3 million in 2025 after hitting 108.1 the year before. That was kind of surprising to me. Maybe there's a limit to how much domestic traffic ATL can pull.

LAX also went down in 2025, from 76.6 to 73.7 million. Not sure what's going on there exactly, could be airlines moving routes or just demand changing. But Heathrow kept climbing and ended up at 84.5 million, its all-time best, so it's not like flying is getting less popular everywhere.

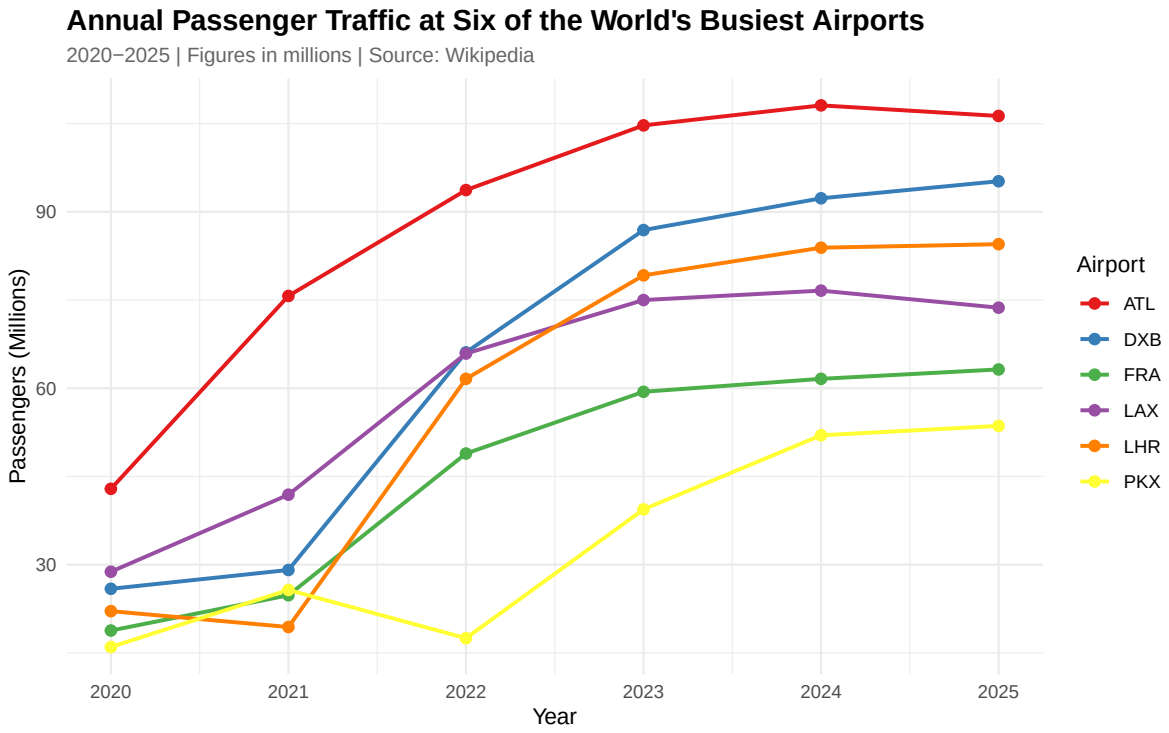


Figure 1: Annual passenger traffic at six of the world's busiest airports, 2020–2025. Each line represents one airport; figures in millions of passengers. Source: Wikipedia.

The weirdest trend in the data is PKX. Every other airport was going up in 2021 and 2022 but Beijing Daxing actually went DOWN from 25.7 to 17.5 million in 2022. That makes no sense until you remember China had those super strict zero-COVID lockdowns. On the chart PKX's line dips when everyone else is climbing, which makes it really stand out. After lockdowns ended it bounced back pretty fast and hit 53.6 million by 2025, though that's still way lower than the other airports.

Then there's Frankfurt which is just kind of slowly chugging along. It got to 63.2 million in 2025 but that's still roughly 10 percent below pre-pandemic levels. It doesn't look like a slow recovery that will eventually catch up—feels more like the airport is just operating at a lower baseline now compared to before.

2 Monte Carlo Numerical Integration

2.1 Method Overview

Monte Carlo integration is a way to estimate the area under a curve using random sampling. The basic idea is you draw a rectangle around the function, throw a bunch of random points at it, and see what fraction end up below the curve. Then you multiply that fraction by the rectangle's area and that gives you an estimate of the integral. The more points you use the better the estimate gets.

I'm working with the Chi-squared PDF with df = 5. The integration domain is $0 \leq x \leq 20$ with $y \in [0, 0.16]$, so the bounding rectangle has area $20 \times 0.16 = 3.2$. I wrote `generate_mcPoints()` to create the random sample, and then a separate function that makes the plot and colors the points depending on whether they're above or below the curve.

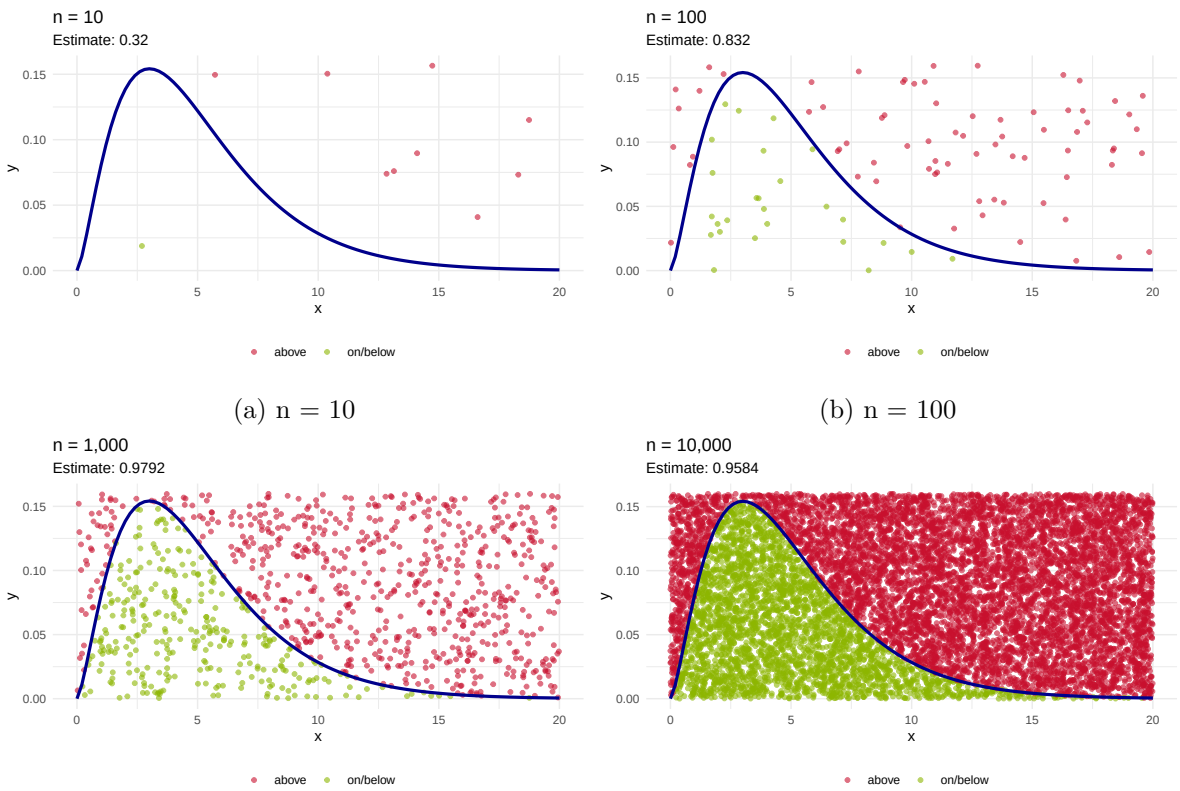
2.2 Small Multiples Visualization

2.3 Analysis

Figure 2 shows the Monte Carlo simulation at four different sample sizes: 10, 100, 1,000, and 10,000. In each panel the dark blue curve is the Chi-squared density (df = 5), green dots are the ones that ended up on or below the curve, and red dots are the ones above. The estimated integral is in the subtitle.

With just 10 points (Figure 2a) the estimate is basically meaningless. There's so few points that the result is almost random and you can't see the curve shape at all. At n = 100 (Figure 2b) you start to get a rough outline of where the curve is, but it's still pretty noisy and the estimate isn't great. Once you get to n = 1,000 (Figure 2c) though it gets way better—the boundary between green and red points actually follows the curve shape and the estimate is getting close to the right answer. At n = 10,000 (Figure 2d) the green region fills in the area under the curve almost perfectly and the estimate is very close to 1.0.

It makes sense that the estimate approaches 1.0 because the Chi-squared PDF (df = 5) is a probability density function, and any proper PDF integrates to exactly 1 over its whole domain. Since the PDF is basically zero past x = 20 for df = 5, our rectangle captures essentially all of the area. So as we throw more points in there, the proportion that falls below the curve times 3.2 should converge on 1.0. You can actually see this happening as you look from the top-left panel to the bottom-right—the estimates get closer and closer, which is pretty much the whole point of the small multiples layout.



(a) n = 10

(b) n = 100

(c) n = 1,000

(d) n = 10,000

Figure 2: Monte Carlo integration of the Chi-squared PDF (df = 5) at four sample sizes. Green points fall on or below the density curve; red points fall above it.

3 Planning and Prompting GenAI Tools

3.1 Plan

3.1.1 Goals

The goal is to take the calcium.csv file, tidy and wrangle the data into a usable format, then produce graphs that help visualize what's going on with calcium levels across the two treatment groups. I also need to write a narrative explaining the results. After I finish the data work and visualization myself, I'll use Claude to generate another version based on a plan-informed prompt, and then compare that output with the generic prompt results provided by the instructor.

3.1.2 Needs

Nouns (things I need): - Laptop/PC, R studio, R environment - The raw data from the calcium.csv file - A tidied and wrangled version of the calcium data - Two visualizations: a group means plot and an individual trajectories plot - A narrative analyzing and comparing the results

Verbs (actions I need to do): - Open and check R studio - Read and inspect the raw CSV - Split the columns into the two treatment groups - Tidy and wrangle the data into long format - Visualize the data with clear, labeled plots - Analyze and compare the plan-informed vs. generic results - Reviewing and Testing

3.1.3 Steps

1. Open the computer and R studio, check the R IDE environment is available
2. Split the eight columns into two groups, columns 1-4 are the null treatment group and columns 5-8 are the diet/exercise group.
3. Rename each column to make the names clear (Year 0, Year 1, Year 2, Year 3 instead of the duplicated y0, y1, y2, y3).
4. Add subject IDs and group labels to each row so I can tell which participant belongs to which group.
5. There is one missing row in the null group (only 15 women, so row 16 is blank), so remove rows where all calcium values are NA.
6. Pivot each group from wide to long format using `pivot_longer()` so that each row is one observation.
7. Combine the two groups into a single tidy data frame with `bind_rows()`.
8. **Review:** Check the final data to make sure it is actually tidy and wrangled correctly—should have around 120 rows ($15 \times 4 + 16 \times 4 = 124$ minus NAs).

9. Use the tidied data to create two graphs: a group means line plot and an individual trajectory plot, both with clear labels and correct data.
10. **Test:** Make sure the graphs look right—proper axis labels, correct group colors, reasonable y-axis range, and that the data points match the table. If something looks off, go back and check the data wrangling steps.

3.1.4 Tidied and Wrangled Data

Table 2: Tidied and Wrangled Calcium Data (separated by group)

Group	Subject	Year 0	Year 1	Year 2	Year 3
Null Treatment					
Null Treatment	Participant 1	87.3	86.9	86.7	75.5
Null Treatment	Participant 2	59.0	60.2	60.0	53.6
Null Treatment	Participant 3	76.7	76.5	75.7	69.5
Null Treatment	Participant 4	70.6	76.1	72.1	65.3
Null Treatment	Participant 5	54.9	55.1	57.2	49.0
Null Treatment	Participant 6	78.2	75.3	69.1	67.6
Null Treatment	Participant 7	73.7	70.8	71.8	74.6
Null Treatment	Participant 8	61.8	68.7	68.2	57.4
Null Treatment	Participant 9	85.3	84.4	79.2	67.0
Null Treatment	Participant 10	82.3	86.9	79.4	77.4
Null Treatment	Participant 11	68.6	65.4	72.3	60.8
Null Treatment	Participant 12	67.8	69.2	66.3	57.9
Null Treatment	Participant 13	66.2	67.0	67.0	56.2
Null Treatment	Participant 14	81.0	82.3	86.8	73.9
Null Treatment	Participant 15	72.3	74.6	75.3	66.1
Diet/Exercise					
Diet/Exercise	Participant 1	83.8	85.5	86.2	81.2
Diet/Exercise	Participant 2	65.3	66.9	67.0	60.6
Diet/Exercise	Participant 3	81.2	79.5	84.5	75.2
Diet/Exercise	Participant 4	75.4	76.7	74.3	66.7
Diet/Exercise	Participant 5	55.3	58.3	59.1	54.2
Diet/Exercise	Participant 6	70.3	72.3	70.6	68.6
Diet/Exercise	Participant 7	76.5	79.9	80.4	71.6
Diet/Exercise	Participant 8	66.0	70.9	70.3	64.1
Diet/Exercise	Participant 9	76.7	79.0	76.9	70.3
Diet/Exercise	Participant 10	77.2	74.0	77.8	67.9
Diet/Exercise	Participant 11	67.3	70.7	68.9	65.9
Diet/Exercise	Participant 12	50.3	51.4	53.6	48.0
Diet/Exercise	Participant 13	57.7	57.0	57.5	51.5
Diet/Exercise	Participant 14	74.3	77.7	72.6	68.0
Diet/Exercise	Participant 15	74.0	74.7	74.5	65.7
Diet/Exercise	Participant 16	57.3	56.0	64.7	53.0

As shown in Table 2, the participants are separated into the null treatment group and the diet/exercise group. There are four measurement time points: "Year 0" is the initial mea-

surement, then Year 1, Year 2, and Year 3. The null treatment group has 15 women and the diet/exercise group has 16 women.

3.2 Plan-Informed Results

I gave my tidied calcium data and my plan to Claude (Claude Sonnet 4.6) on April 13, 2026. Here's the prompt I used:

> Based on my tidied calcium data with two groups (Null Treatment with 15 women, and Diet/Exercise with 16 women) measured at four time points (Year 0 through Year 3), I need two line charts: 1. a group means plot showing the average calcium level for each group over time, with the mean values labeled on each point, and 2. an individual lines plot showing each woman's trajectory, faceted by group. Use ggplot2 in R.

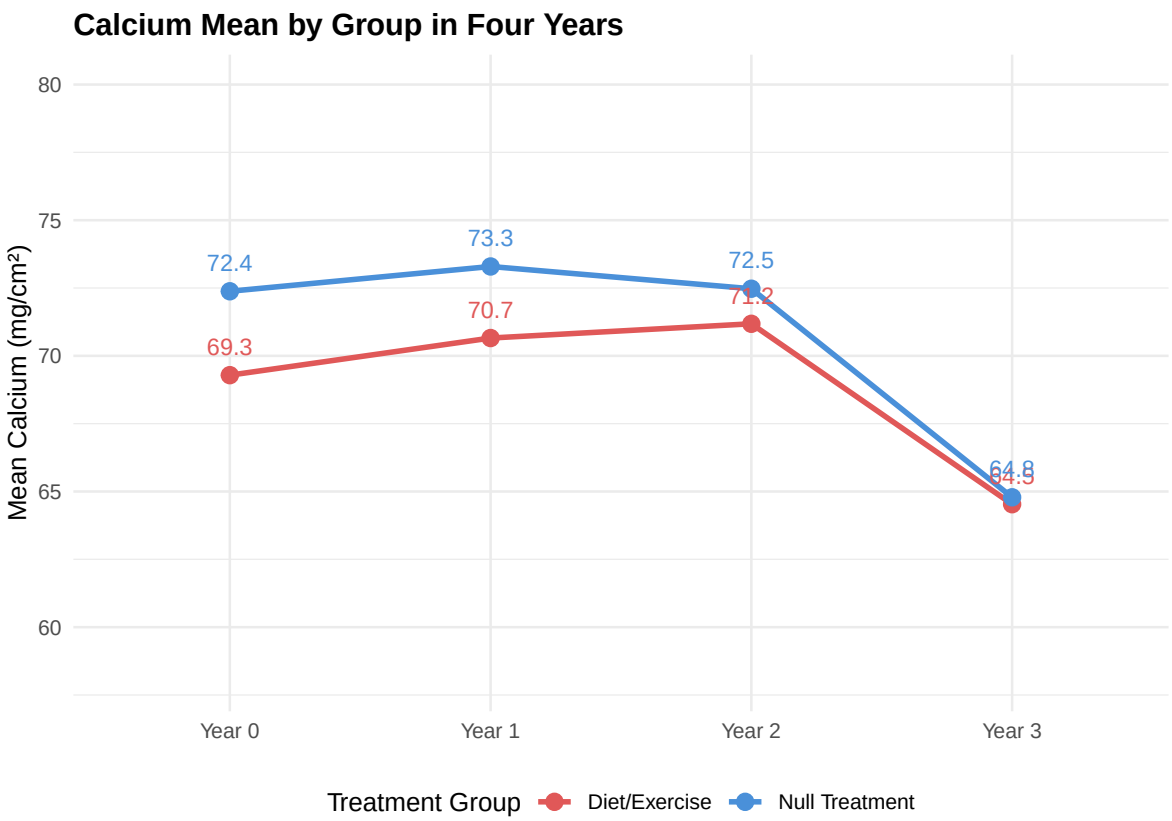


Figure 3: Calcium mean by group over four years. Generated by Claude using a plan-informed prompt.

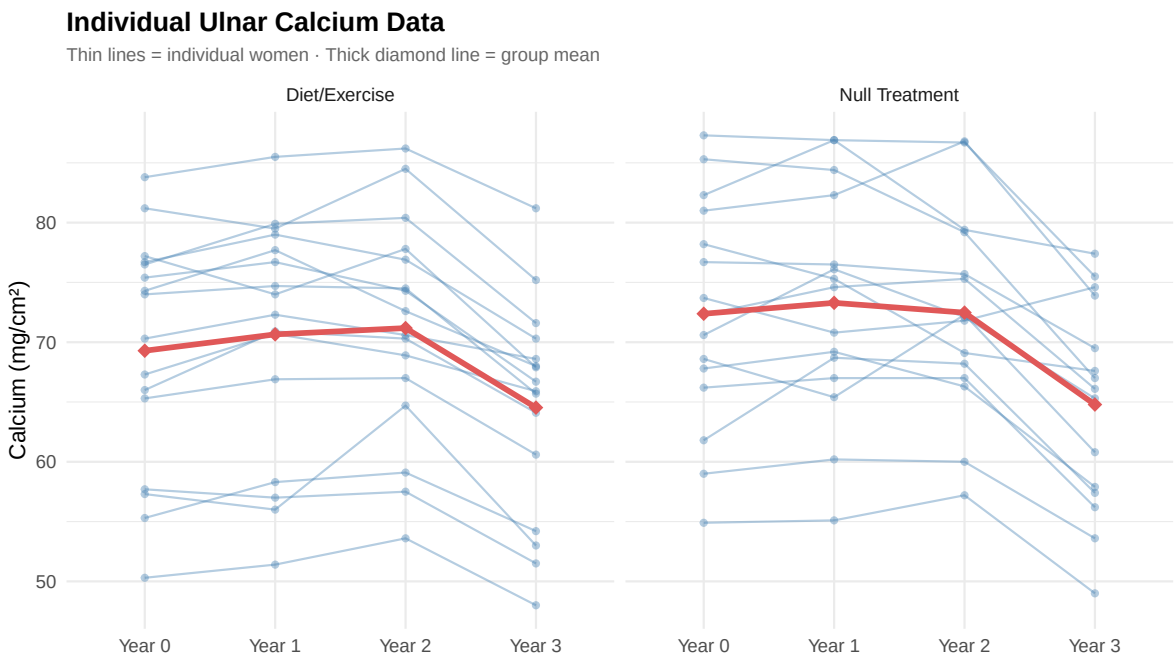


Figure 4: Individual ulnar calcium data. Thin lines show individual women; thick diamond line shows the group mean. Generated by Claude using a plan-informed prompt.

Based on my plan and the prompt I showed above, Claude gave me Figure 3 and Figure 4.

Looking at Figure 3, the mean calcium content of the two groups doesn't have a huge difference overall, but there are some small differences at each time point. Both groups show a similar pattern: calcium goes up a bit from Year 0 to Year 1, and then drops pretty significantly from Year 2 to Year 3. The difference between them is that the null treatment group's calcium dips slightly from Year 1 to Year 2, while the diet/exercise group's calcium actually goes up a little in that same period. Overall though, the null treatment group's mean calcium is consistently a bit higher than the diet/exercise group.

Looking at Figure 4, it's immediately obvious that individual variation is really big. Some women's calcium changes a ton from year to year, while others barely move. Some women start with high calcium and stay high, others start low and stay low. This tells me that just looking at the group means would be kind of misleading because there's so much person-to-person variation being averaged out. When I look at both graphs together, the intervention effect of the diet/exercise program doesn't seem super significant or consistent. It helped some women increase or keep their calcium levels steady in the first couple years, but over the full four-year period, both groups ended up trending in roughly the same direction. That big drop in Year 3 happened in both groups, which seems like it might just be a natural aging effect rather than anything related to the treatment.

3.3 Generic Prompt Results

These two figures are from the instructor-provided generic prompt output. The instructor gave Claude Sonnet 4.6 Extended (March 31, 2026) a simple generic prompt about the calcium data without any specific plan or guidance about the data structure.

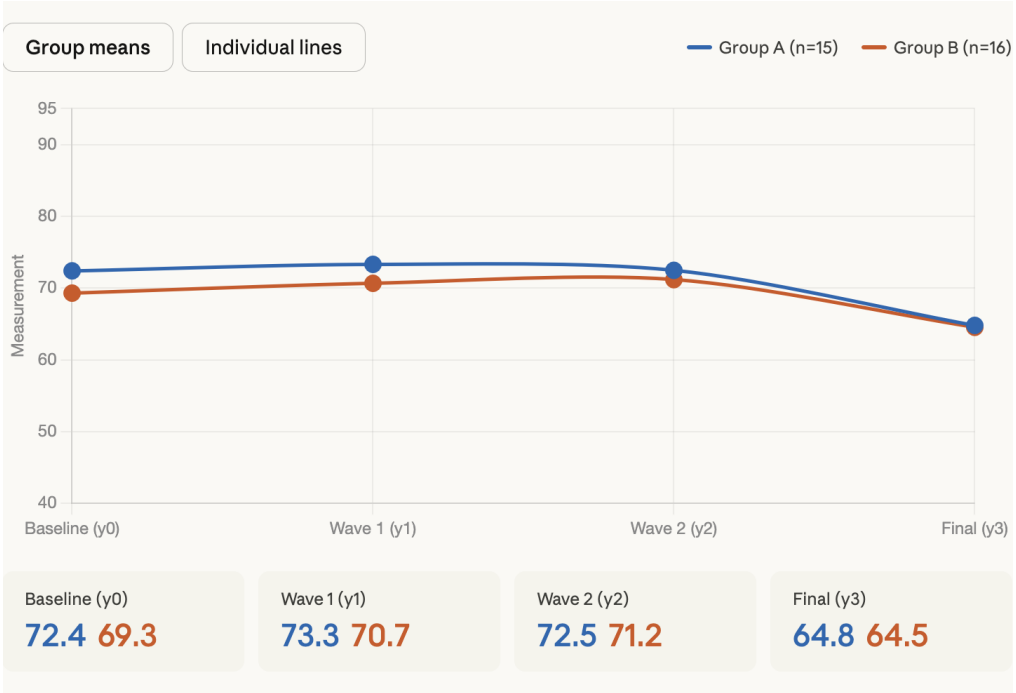


Figure 5: Generic prompt output: group means over time. Produced by Claude Sonnet 4.6 Extended using an unguided prompt provided by the instructor.

3.4 Comparison: Plan-Informed vs. Generic

Comparing the plan-informed results with the generic output, there are some clear differences.

The plan-informed output (Figure 3 and Figure 4) produced exactly the two charts I asked for, with proper group labels ("Null Treatment" and "Diet/Exercise"), actual mean values labeled on the data points, and a faceted individual plot that overlays the group mean as a thicker line. Because I told Claude about the data structure in my prompt—like which columns belong to which group and how many women are in each—the code handled the tricky tidying parts correctly without me having to fix anything.

The generic output (Figure 5 and Figure 6) managed to make two charts as well, which is decent. But the group means plot just uses generic labels like "Group A" and "Group B"

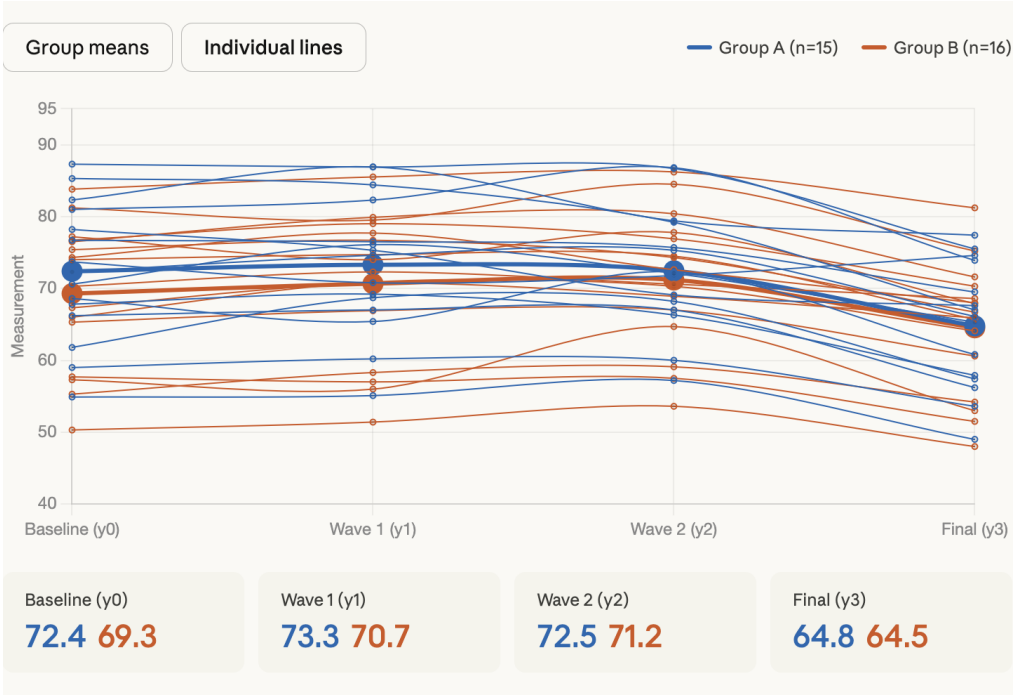


Figure 6: Generic prompt output: individual trajectories. Produced by Claude Sonnet 4.6 Extended using an unguided prompt provided by the instructor.

instead of the actual treatment names, so you have to figure out which is which yourself. There's also no data labels on the points and no error bars, so you can't really tell how precise those means are. The individual trajectories plot from the generic output does look pretty good honestly, but it's clear that without specific guidance about the data format, the generic prompt missed some details that would've made the results more useful right away.

Overall the plan-informed approach saved me time because I didn't need to go back and fix labels or add missing features. The generic version gave a reasonable starting point but I would've had to do a bunch of edits to make it presentation-ready. This pretty much confirms what we learned in class about how the quality of AI output depends a lot on the quality of the input you give it.

4 Reflection

This assignment ended up being a lot more work than I expected but I learned more than I thought I would from doing it.

The airport section was solid practice for data wrangling and visualization. Building the tibble, pivoting, making a line chart—I've done all of that before. But writing the analysis was harder than I thought it would be. I had to actually sit with Table 1 and Figure 1 and think about what the trends meant. Like I didn't realize ATL and LAX both went down in 2025 until I had the chart in front of me, and the whole PKX zero-COVID dip was something I only caught because I was comparing across all six airports at once. Having the table and chart together made patterns way easier to spot than either one would alone.

The Monte Carlo section was probably my favorite part. I knew the concept from lecture but coding it up myself and seeing the estimates get better from 10 to 10,000 points in Figure 2 was a different kind of understanding. Writing the `generate_mcPoints()` function and using `set.seed()` for reproducibility were useful skills. Figuring out the Quarto layout div for the small multiples was kind of frustrating at first but the end result looks clean.

Section 3 with the GenAI comparison was really where things clicked for me. My plan-informed prompt gave me plots that had proper group names, data labels, and a clean faceted layout. The generic prompt from the instructor gave decent charts but with generic labels and no error indicators. That difference showed me that planning what you want before talking to the AI actually matters a lot. It's like the "garbage in, garbage out" idea but for prompts—if you give the AI a vague request you get vague results.

Putting the whole thing together in Quarto was also new for me. Cross-referencing figures and tables with `@fig-` and `@tbl-` labels and having everything render from one source file is really useful for reproducibility. I can see how this would be valuable for actual research papers later on.

Appendix A: GenAI Usage

This appendix documents all uses of generative AI tools in this assignment, following the course academic integrity policy.

Usage 1: Plan-Informed Calcium Visualization (Section 3)

- **Tool:** Claude Sonnet 4.6 (Anthropic)
- **Date:** April 13, 2026
- **Purpose:** Generate R code to visualize the tidied calcium data based on my pre-written plan.
- **Exact prompt used:**

  > Based on my tidied calcium data with two groups (Null Treatment with 15 women, and Diet/Exercise with 16 women) measured at four time points (Year 0 through Year 3), I need two line charts: 1. a group means plot showing the average calcium level for each group over time, with the mean values labeled on each point, and 2. an individual lines plot showing each woman's trajectory, faceted by group. Use ggplot2 in R.

- **Exact response generated by Claude:** Claude returned R code that used `group_by()` and `summarise()` to compute group means, SD, and SE, then produced two ggplot2 charts. The first chart was a group means line plot using `geom_line()` + `geom_point()` + `geom_text()` for labeling mean values, with `scale_color_manual()` to set blue for Null Treatment and red for Diet/Exercise. The second chart used `geom_line()` and `geom_point()` for individual trajectories, `facet_wrap(~ Group)` to separate the groups, and overlaid the group mean as a thicker line with diamond-shaped points using `shape = 18`. Both charts used `theme_minimal()` for styling. The generated code is shown in the plan-informed results chunks in Section 3 and produced the plots in Figure 3 and Figure 4.

Usage 2: Generic Prompt Output (Instructor-Provided)

- **Tool:** Claude Sonnet 4.6 Extended (Anthropic)
- **Date:** March 31, 2026
- **Purpose:** Instructor-provided baseline for comparison in the GenAI prompting exercise.
- **Prompt:** Generic prompt provided by instructor (not authored by student).
- **Response:** Two visualizations: a group means line plot (Figure 5) and an individual trajectories plot (Figure 6). These outputs were provided as-is by the instructor for comparison purposes.

Appendix B: Code

All R code used in this document is shown below.

```
# ============================================================
# SECTION 1: BUSIEST AIRPORTS ANALYSIS
# ============================================================

# --- Airport Data ---
airport_data <- tibble(
  airport = rep(c("ATL", "FRA", "PKX", "DXB", "LAX", "LHR"), each = 6),
  name = rep(c("Hartsfield-Jackson Atlanta", "Frankfurt Airport",
               "Beijing Daxing", "Dubai International",
               "Los Angeles International", "London Heathrow"), each = 6),
  year = rep(2020:2025, times = 6),
  passengers_M = c(
    42.9, 75.7, 93.7, 104.7, 108.1, 106.3,
    18.8, 24.8, 48.9, 59.4,  61.6,  63.2,
    16.0, 25.7, 17.5, 39.4,  52.0,  53.6,
    25.9, 29.1, 66.1, 86.9,  92.3,  95.2,
    28.8, 41.9, 65.9, 75.0,  76.6,  73.7,
    22.1, 19.4, 61.6, 79.2,  83.9,  84.5
  )
)

# --- Airport Table ---
airport_data |>
  select(airport, name, year, passengers_M) |>
  pivot_wider(names_from = year, values_from = passengers_M) |>
  rename(Airport = airport, Name = name) |>
  kable(digits = 1, align = c("l", "l", rep("r", 6))) |>
  kable_styling(latex_options = c("hold_position", "scale_down"),
                font_size = 10) |>
  column_spec(1, bold = TRUE)

# --- Airport Plot ---
airport_data |>
  filter(!is.na(passengers_M)) |>
  ggplot(aes(x = year, y = passengers_M, color = airport)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2.2) +
  scale_color_brewer(palette = "Set1") +
```

```r
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(breaks = seq(0, 120, by = 30)) +
  labs(
    title = paste0("Annual Passenger Traffic at Six of the ",
                   "World's Busiest Airports"),
    subtitle = "2020\u20132025 | Figures in millions | Source: Wikipedia",
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    plot.subtitle = element_text(size = 10, color = "gray40"),
    legend.position = "right"
  )

# ============================================================
# SECTION 2: MONTE CARLO NUMERICAL INTEGRATION
# ============================================================

# --- Helper Functions ---
generate_mcPoints <- function(n, x_min, x_max, y_min, y_max) {
  data.frame(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

make_mc_plot <- function(n, seed_val) {
  set.seed(seed_val)
  mc_data <- generate_mcPoints(n, 0, 20, 0, 0.16) |>
    mutate(flag = if_else(y > dchisq(x, df = 5), "above", "on/below"))

  rect_area <- 20 * 0.16
  estimate  <- round(rect_area * mean(mc_data$flag == "on/below"), 4)

  ggplot(mc_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.6, size = 1.2) +
    stat_function(fun = dchisq, args = list(df = 5), xlim = c(0, 20),
                  color = "darkblue", linewidth = 1) +
    scale_color_manual(values = c("above" = "#C8102E",
```

```r
                                  "on/below" = "#8DB600")) +
    labs(
      title = paste0("n = ", format(n, big.mark = ",")),
      subtitle = paste0("Estimate: ", estimate),
      x = "x", y = "y", color = NULL
    ) +
    theme_minimal(base_size = 10) +
    theme(legend.position = "bottom")
}

# --- Monte Carlo Plots (called inside Quarto layout div) ---
# fig-mc-n10:    make_mc_plot(10, seed_val = 42)
# fig-mc-n100:   make_mc_plot(100, seed_val = 99)
# fig-mc-n1000:  make_mc_plot(1000, seed_val = 7)
# fig-mc-n10000: make_mc_plot(10000, seed_val = 21)

# ============================================================
# SECTION 3: CALCIUM DATA ANALYSIS
# ============================================================

# --- Read and Tidy the Calcium Data ---
calcium_raw <- read_csv("calcium.csv", show_col_types = FALSE)

null_group <- calcium_raw |>
  select(1:4) |>
  set_names(c("Year 0", "Year 1", "Year 2", "Year 3")) |>
  mutate(Subject = paste0("Participant ", row_number()),
         Group = "Null Treatment") |>
  filter(!if_all(`Year 0`:`Year 3`, is.na))

exercise_group <- calcium_raw |>
  select(5:8) |>
  set_names(c("Year 0", "Year 1", "Year 2", "Year 3")) |>
  mutate(Subject = paste0("Participant ", row_number()),
         Group = "Diet/Exercise")

calcium_wide <- bind_rows(null_group, exercise_group)

calcium_tidy <- calcium_wide |>
  pivot_longer(cols = `Year 0`:`Year 3`,
               names_to = "TimePoint",
               values_to = "Calcium") |>
```

```r
  mutate(TimePoint = factor(TimePoint,
                            levels = c("Year 0", "Year 1",
                                       "Year 2", "Year 3"))) |>
  drop_na(Calcium)

# --- Tidied Data Table ---
calcium_wide |>
  select(Group, Subject, `Year 0`, `Year 1`, `Year 2`, `Year 3`) |>
  kable(digits = 1, align = c("l", "l", rep("r", 4))) |>
  kable_styling(latex_options = c("hold_position", "scale_down"),
                font_size = 9) |>
  column_spec(1, bold = TRUE) |>
  pack_rows("Null Treatment",
            start_row = 1,
            end_row = sum(calcium_wide$Group == "Null Treatment")) |>
  pack_rows("Diet/Exercise",
            start_row = sum(calcium_wide$Group == "Null Treatment") + 1,
            end_row = nrow(calcium_wide))

# --- Summary Stats ---
calcium_summary <- calcium_tidy |>
  group_by(Group, TimePoint) |>
  summarise(
    Mean = mean(Calcium),
    SD   = sd(Calcium),
    SE   = SD / sqrt(n()),
    .groups = "drop"
  )

# --- Plan-Informed Plot 1: Group Means ---
ggplot(calcium_summary, aes(x = TimePoint, y = Mean,
                            color = Group, group = Group)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  geom_text(aes(label = round(Mean, 1)),
            vjust = -1.2, size = 3.5, show.legend = FALSE) +
  scale_color_manual(values = c("Diet/Exercise" = "#E05858",
                                "Null Treatment" = "#4A90D9"),
                     name = "Treatment Group") +
  scale_y_continuous(limits = c(58, 80)) +
  labs(
    title = "Calcium Mean by Group in Four Years",
```

```r
    x = NULL,
    y = "Mean Calcium (mg/cm\u00b2)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 13),
    legend.position = "bottom"
  )

# --- Plan-Informed Plot 2: Individual Trajectories ---
ggplot(calcium_tidy, aes(x = TimePoint, y = Calcium, group = Subject)) +
  geom_line(alpha = 0.4, linewidth = 0.5, color = "steelblue") +
  geom_point(alpha = 0.4, size = 1.2, color = "steelblue") +
  geom_line(data = calcium_summary,
            aes(x = TimePoint, y = Mean, group = Group),
            linewidth = 1.3, color = "#E05858") +
  geom_point(data = calcium_summary,
             aes(x = TimePoint, y = Mean, group = Group),
             size = 3, shape = 18, color = "#E05858") +
  facet_wrap(~ Group) +
  labs(
    title = "Individual Ulnar Calcium Data",
    subtitle = "Thin lines = individual women \u00b7 Thick diamond line = group mean",
    x = NULL,
    y = "Calcium (mg/cm\u00b2)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "gray40")
  )

# --- Generic Prompt Images ---
# knitr::include_graphics("generic_group_means.png")
# knitr::include_graphics("generic_individual_lines.png")
```